

true

Scaling Deep Research Automation with Multi-Agent Workflow

Building an Agentic Fabric Using DOVA, Strands Agents SDK, and Amazon Bedrock AgentCore

DOVA Team

Introduction

Organizations increasingly rely on AI-powered research to stay competitive—whether analyzing market trends, evaluating emerging technologies, or synthesizing findings from academic literature. However, traditional single-agent AI systems struggle with complex research tasks that require diverse perspectives, source verification, and iterative refinement.

In this post, we introduce **DOVA (Deep Orchestrated Versatile Agent)**, an open-source multi-agent framework built on **Amazon Strands Agents SDK** and **Amazon Bedrock AgentCore** that transforms how teams conduct automated research. By orchestrating specialized agents through collaborative reasoning patterns, DOVA achieves research quality that exceeds what any single agent can produce alone.

The Challenge: Limitations of Single-Agent Research

Current Bottlenecks

Single-agent AI systems face several fundamental limitations when handling complex research tasks:

- **Linear thinking:** Agents process information sequentially, missing connections across domains
- **Premature conclusions:** Without challenge mechanisms, agents accept initial findings without verification
- **Limited perspectives:** A single model brings one interpretation to ambiguous data
- **No self-correction:** Errors in early reasoning steps propagate through the entire analysis

Scale and Quality Trade-offs

Research teams face a difficult choice: invest significant human effort for thorough analysis, or accept shallow AI-generated summaries. Neither option scales effectively for organizations processing hundreds of research queries across domains like academic papers, code repositories, and market reports.

Solution Overview: The Agentic Fabric Architecture

DOVA addresses these challenges through an **Agentic Fabric**—a dynamic multi-agent system where specialized agents collaborate through structured reasoning patterns, powered by AWS managed services.

AWS Technology Stack

Technology	Purpose	Key Features
Strands Agents SDK	Agentic platform & orchestration	Native MCP support, multi-agent patterns (Swarm, Graph), model-agnostic, 40+ built-in tools
Amazon Bedrock AgentCore	Managed agent services	Serverless runtime, persistent memory, secure gateway, code interpreter
Amazon Bedrock	Foundation models	Claude, Nova, Titan for inference at scale

Architecture Components

Component	Function	AWS Service
Agent Runtime	Serverless agent deployment with auto-scaling	AgentCore Runtime
MCP Gateway	Authenticated tool integration for external sources	AgentCore Gateway
User Memory	Persistent memory across sessions (preferences, facts, summaries)	AgentCore Memory
Code Sandbox	Secure execution environment for validation	AgentCore Code Interpreter
Observability	Metrics, traces, logs, and alarms	CloudWatch + X-Ray + OpenTelemetry
Identity & Security	SSO/OAuth authentication and authorization	AWS Cognito + AgentCore Gateway

Agent Specializations

The fabric comprises five specialized agents built using the Strands Agents SDK:

Agent	Responsibility	Strands Pattern
Research Agent	Deep source traversal via MCP servers (ArXiv, GitHub, HuggingFace)	Agent with MCPClient tools
Profiling Agent	Adapts findings using AgentCore Memory	Agent with memory tools
Validation Agent	Executes code in AgentCore Code Interpreter	Agent with code_interpreter tool
Synthesis Agent	Integrates findings into coherent narratives	Agent with RAG tools
Dialectic Agent	Adversarial debate using Swarm pattern	Swarm with opposing prompts

Technical Implementation

Component 1: Research Agent with Native MCP Support

The **Strands Agents SDK** provides first-class MCP (Model Context Protocol) integration for connecting to external data sources.

```
from strands import Agent
from strands.tools.mcp import MCPClient
from mcp import stdio_client, StdioServerParameters

class DOVAResearchAgent:
    """Multi-source research agent using Strands SDK + MCP servers."""

    def __init__(self):
        # Initialize MCP clients for each data source
        self.arxiv_mcp = MCPClient(
            lambda: stdio_client(StdioServerParameters(
                command="uvx",
                args=["mcp-server-arxiv"]
            ))
        )

        self.github_mcp = MCPClient(
            lambda: stdio_client(StdioServerParameters(
                command="npx",
                args=["-y", "@modelcontextprotocol/server-github"]
            ))
        )

        self.hf_mcp = MCPClient(
```

```

        lambda: stdio_client(StdioServerParameters(
            command="uvx",
            args=["mcp-server-huggingface"]
        ))
    )

def create_agent(self) -> Agent:
    """Create the research agent with all MCP tools."""
    all_tools = []

    # Gather tools from all MCP servers
    with self.arxiv_mcp:
        all_tools.extend(self.arxiv_mcp.list_tools_sync())

    with self.github_mcp:
        all_tools.extend(self.github_mcp.list_tools_sync())

    with self.hf_mcp:
        all_tools.extend(self.hf_mcp.list_tools_sync())

    return Agent(
        system_prompt="""You are DOVA, a Deep Orchestrated Versatile Agent.
        Search multiple sources in parallel, cross-reference findings,
        and synthesize into coherent summaries with citations."""
        , tools=all_tools
    )

```

Key implementation detail: The Strands SDK's `MCPClient` wraps MCP server connections, enabling agents to invoke tools from ArXiv, GitHub, and HuggingFace through a unified interface.

Component 2: User Profiling with AgentCore Memory

AgentCore Memory provides built-in memory strategies for persistent user context:

```

from strands import Agent
from strands_tools.agent_core_memory import AgentCoreMemoryToolProvider
from dataclasses import dataclass

@dataclass
class UserProfile:
    user_id: str
    session_id: str
    namespace: str = "dova_profiles"

class DOVAProfileAgent:
    """User profiling agent using AgentCore Memory."""

    def __init__(self, memory_id: str, region: str = "us-west-2"):
        self.memory_id = memory_id
        self.region = region

    def create_agent(self, profile: UserProfile) -> Agent:

```

```

"""Create a profiling agent for a specific user."""
# Initialize AgentCore Memory provider per-request
memory_provider = AgentCoreMemoryToolProvider(
    memory_id=self.memory_id,
    actor_id=profile.user_id,
    session_id=profile.session_id,
    namespace=profile.namespace,
    region=self.region
)

return Agent(
    system_prompt="""You are the DOVA User Profiling Agent.
Your responsibilities:
1. RECORD user preferences, interests, and expertise levels
2. RETRIEVE relevant user context for personalization
3. UPDATE user profiles based on interactions

Memory Actions:
- action="record": Store new information about the user
- action="retrieve": Search for relevant user context
- action="list": List all stored memories
""",
    tools=memory_provider.tools
)

async def get_user_context(self, profile: UserProfile, query: str) -> str:
    """Retrieve relevant user context for a query."""
    agent = self.create_agent(profile)
    result = agent(f"Retrieve any relevant user preferences for: {query}")
    return result.message

```

Memory strategies (built into AgentCore): - **User Preferences**: Research domains, output format preferences, notification settings - **Semantic Facts**: Learned expertise areas, saved searches, bookmarked papers - **Session Summaries**: Conversation context for multi-turn research sessions

Component 3: Code Validation with AgentCore Code Interpreter

The **AgentCore Code Interpreter** provides a secure, managed sandbox for executing code claims.

```

from strands import Agent
from strands_tools.code_interpreter import AgentCoreCodeInterpreter

class DOVAValidationAgent:
    """Code validation agent using AgentCore Code Interpreter."""

    def __init__(self, region: str = "us-west-2"):
        self.code_interpreter = AgentCoreCodeInterpreter(region=region)

    def create_agent(self) -> Agent:
        """Create the validation agent."""

```

```

return Agent(
    system_prompt="""You are the DOVA Validation Agent.
    Your responsibilities:
    1. VALIDATE code snippets from research papers
    2. TEST implementations for correctness
    3. BENCHMARK performance when relevant
    4. REPORT issues and suggestions

    When validating code:
    - First create a session for the validation task
    - Execute the code in the sandbox
    - Capture outputs, errors, and metrics
    - Provide clear pass/fail assessment with reasoning
    """,
    tools=[self.code_interpreter.code_interpreter]
)

async def validate_snippet(
    self,
    code: str,
    language: str = "python",
    description: str = ""
) -> dict:
    """Validate a code snippet."""
    agent = self.create_agent()
    result = agent(f"""
    Please validate this {language} code:

    Description: {description}

    ```{language}
 {code}
    ```

    Steps:
    1. Create a validation session
    2. Execute the code
    3. Report results (success/failure, output, any errors)
    """)
    return {
        "validation_result": result.message,
        "success": "error" not in result.message.lower()
    }

```

Security: AgentCore Code Interpreter runs in isolated containers with resource limits, network restrictions, and automatic cleanup.

Component 4: Master Orchestrator with Multi-Agent Coordination

The **Strands SDK** provides built-in multi-agent patterns for orchestration.

```

from strands import Agent, tool
from strands_tools import batch
from typing import List, Dict, Any

```

```

class DOVAOrchestrator:
    """Master orchestrator using Strands multi-agent patterns."""

    def __init__(
        self,
        research_agent: Agent,
        profiling_agent: Agent,
        validation_agent: Agent
    ):
        self.research_agent = research_agent
        self.profilng_agent = profiling_agent
        self.validation_agent = validation_agent
        self._setup_agent_tools()

    def _setup_agent_tools(self):
        """Wrap sub-agents as tools for the orchestrator."""

        @tool
        def search_research(query: str) -> str:
            """Search for research papers, code, and models."""
            return self.research_agent(query).message

        @tool
        def get_user_context(user_id: str, query: str) -> str:
            """Retrieve user profile and preferences."""
            return self.profilng_agent(
                f"Get context for user {user_id} regarding: {query}"
            ).message

        @tool
        def validate_code(code: str, description: str) -> str:
            """Validate code in a secure sandbox environment."""
            return self.validation_agent(
                f"Validate this code:\n```\n{code}\n```\nDescription: {description}"
            ).message

        @tool
        def synthesize_findings(findings: List[str]) -> str:
            """Synthesize multiple research findings into a summary."""
            combined = "\n\n".join(findings)
            return self.research_agent(
                f"Synthesize these findings:\n{combined}"
            ).message

    self.tools = [
        search_research,
        get_user_context,
        validate_code,
        synthesize_findings,
        batch # For parallel execution of independent tasks
    ]

    def create_orchestrator(self) -> Agent:
        """Create the master orchestrator agent."""
        return Agent(
            system_prompt="""You are the DOVA Master Orchestrator.

            Your role is to:
            1. UNDERSTAND user intent and decompose complex queries
            2. COORDINATE multiple specialized agents
            3. PERSONALIZE responses based on user context
            4. SYNTHESIZE findings from multiple sources
            5. VALIDATE code and implementations when needed

            Use the batch tool to execute independent tasks in parallel.
            Always cite sources and provide actionable insights."""

```



```

        tools=self.tools
    )

    async def process_query(self, user_id: str, query: str) -> Dict[str, Any]:
        """Process a user query through the orchestration pipeline."""
        orchestrator = self.create_orchestrator()
        result = orchestrator(f"""
        User ID: {user_id}
        Query: {query}

        Please:
        1. Get user context first
        2. Search for relevant research
        3. Synthesize personalized findings
        """)
        return {
            "response": result.message,
            "tools_called": len(result.tool_calls) if result.tool_calls else 0
        }

```

Component 5: Collaborative Reasoning Patterns

DOVA implements collaborative patterns on top of Strands primitives:

Blackboard Pattern (async contribution):

```

class Blackboard:
    """Shared workspace for collaborative agent reasoning."""

    async def post(
        self,
        agent_name: str,
        post_type: PostType, # HYPOTHESIS, EVIDENCE, REFINEMENT
        content: str,
        confidence: float = 0.5,
    ) -> str:
        """Post an insight for others to build upon."""
        post = BlackboardPost(
            id=f"post_{uuid4().hex[:8]}",
            agent_name=agent_name,
            post_type=post_type,
            content=content,
            confidence=confidence,
        )
        self._posts[post.id] = post
        return post.id

```

Debate Agents Pattern (Bull vs Bear using Strands `swarm`):

```

from strands import Agent
from strands_tools import swarm

class DOVADebateAgents:

```

```

"""Bull vs Bear debate pattern for balanced analysis."""

def create_advocate_agent(self) -> Agent:
    """Create the Bull (advocate) agent."""
    return Agent(
        system_prompt="""You are the DOVA Advocate Agent (Bull).
        Present the STRONGEST CASE for the proposed solution.
        Focus on: feasibility, innovation, competitive advantages."""
    )

def create_critic_agent(self) -> Agent:
    """Create the Bear (critic) agent."""
    return Agent(
        system_prompt="""You are the DOVA Critic Agent (Bear).
        Present LEGITIMATE CONCERNS about the proposed solution.
        Focus on: risks, costs, complexity, edge cases."""
    )

def create_synthesis_agent(self) -> Agent:
    """Create the synthesis agent for balanced conclusions."""
    return Agent(
        system_prompt="""You are the DOVA Synthesis Agent.
        Given Bull and Bear arguments, provide a balanced assessment.
        Acknowledge valid points from both sides.""",
        tools=[swarm] # Coordinate Bull/Bear in parallel
    )

async def debate(self, topic: str, context: str) -> dict:
    """Run a Bull vs Bear debate on a topic."""
    bull = self.create_advocate_agent()
    bear = self.create_critic_agent()
    synthesis = self.create_synthesis_agent()

    bull_args = bull(f"Argue FOR: {topic}\nContext: {context}").message
    bear_args = bear(f"Argue AGAINST: {topic}\nContext: {context}").message

    final = synthesis(f"""
    Topic: {topic}
    Bull arguments: {bull_args}
    Bear arguments: {bear_args}
    Provide balanced synthesis.
    """)
    return {"bull": bull_args, "bear": bear_args, "synthesis": final.message}

```

Iterative Refinement (serial depth):

```

async def iterative_refinement(
    problem: str,
    agents: list[Agent],
    max_iterations: int = 3
) -> str:
    """Agents take turns critiquing and refining."""
    current_answer = agents[0](problem).message

    for i in range(1, max_iterations):
        agent = agents[i % len(agents)]
        current_answer = agent(
            f"Critique and improve: {current_answer}"
        ).message

    return current_answer

```

Deployment with AgentCore

Project Setup

```
# Clone the fullstack AgentCore template
git clone https://github.com/awslabs/fullstack-solution-template-for-agentcore
cd fullstack-solution-template-for-agentcore

# Deploy infrastructure (Cognito, API Gateway, Frontend)
cd infra-cdk && npm install && cdk bootstrap && cdk deploy

# Install DOVA
pip install strands-agents strands-agents-tools
pip install -e ".[dev]"
```

Deploy to AgentCore Runtime

```
# Install AgentCore toolkit
pip install bedrock-agentcore-starter-toolkit

# Configure agent entry point
agentcore configure -e dova/main.py

# Deploy to AgentCore Runtime (serverless)
agentcore launch

# Test the deployed agent
agentcore invoke '{"query": "Latest advances in multi-agent systems"}'

# View logs and traces
agentcore logs
```

Reasoning Mode Configuration

Mode	Use Case	Strands Pattern	AgentCore Services
quick	Real-time queries	Single Agent	Runtime only
standard	Most requests	Agent + reflection	Runtime + Memory
deep	Complex analysis	Batch + Debate agents	Runtime + Memory + Code Interpreter
collaborative	Research projects	Full orchestration	All services

Observability and Monitoring

DOVA leverages built-in observability through Strands SDK and AWS services.

OpenTelemetry Integration

```
from strands import Agent
from strands.telemetry import configure_telemetry

# Configure OpenTelemetry + CloudWatch
configure_telemetry(
    service_name="dova",
    exporter="cloudwatch", # Also supports: langfuse, jaeger, otlp
)

agent = Agent(system_prompt="Research agent with tracing enabled")
```

CloudWatch Metrics and Alarms

Metric	Description	Alarm Threshold
AgentInvocations	Total agent calls	N/A (monitoring)
AgentLatencyP99	99th percentile latency	> 30s
MCPToolErrors	MCP server failures	> 5% error rate
MemoryOperations	AgentCore Memory calls	N/A (monitoring)
CodeInterpreterExecutions	Sandbox runs	N/A (monitoring)

Results and Benefits

Operational Improvements

- **Research depth:** Multi-agent collaboration surfaces insights missed by single-agent systems
- **Quality assurance:** Adversarial dialectic patterns catch errors before output

- **Auditability:** Full reasoning traces via OpenTelemetry enable review of conclusions
- **Scalability:** AgentCore Runtime auto-scales based on demand
- **Cost efficiency:** Serverless deployment reduces infrastructure overhead by 50-70%

Key Learnings

Conclusion

DOVA demonstrates that multi-agent collaboration, implemented through the **Strands Agents SDK** for orchestration and **Amazon Bedrock AgentCore** for managed services, can significantly improve automated research quality while reducing infrastructure complexity.

The combination of: - **Strands SDK** for native MCP support, multi-agent patterns, and model-agnostic design - **AgentCore Runtime** for serverless deployment - **AgentCore Memory** for persistent user context - **AgentCore Gateway** for secure tool integration - **AgentCore Code Interpreter** for sandboxed validation

...enables teams to deploy sophisticated research automation that adapts to complexity without managing infrastructure.

Learn More

- **Strands Agents SDK:** strandsagents.com
- **Strands GitHub:** github.com/strands-agents/sdk-python
- **Amazon Bedrock AgentCore:** docs.aws.amazon.com/bedrock-agentcore

- **FAST Template:** github.com/aws-labs/fullstack-solution-template-for-agentcore
-